



JSON Schema

Google Summer of Code 2026 Project Proposal

Title: Add Support for Reporting on the
Annotation Test Suite
(Medium: 175 hours)

Mentors:



Contributor:



Table of Contents

1. Cover Page
2. Table of Content
3. About Me
4. Motivation | Why JSON Schema?
5. Contributions
6. Qualification Task
7. Project Description
 - a. Overview
 - b. Project Outcomes
 - c. Implementation Plan
8. Timeline
9. Post GSoC Plans

ABOUT ME

Name [REDACTED]

GitHub Username : [REDACTED]

Email : [REDACTED]

Alternative Mail [REDACTED]

University Name : [REDACTED]

Major : Electrical Engineering

Degree : Bachelor of Technology

Current Year : Sophomore

LinkedIn : [REDACTED]

Country : India

Time Zone : IST (UTC +05:30)

Motivation

Why participating in GSoC?

I am motivated to participate in GSoC because I want to transition from being a consumer of technology to becoming an active contributor. I want to apply my experience in Full-Stack development to help maintain and improve the open-source infrastructure that the developer community relies on and want to give back to the community. For me, GSoC is the perfect opportunity to bridge the gap between building personal projects and contributing to high-impact software under the guidance of world-class mentors.

Why JSON Schema?

There are several reasons why I am choosing JSON Schema as my mentor organization. First and foremost, the project aligns perfectly with my existing tech stack and my focus on building reliable, scalable systems. While exploring various open-source organizations, I was highly impressed by the clarity of the JSON Schema documentation and the depth of its community engagement.

JSON Schema is particularly significant to me as it marks the beginning of my journey into the open-source world. It is the first organization I have dedicated myself to, and over the past three months, I have been actively contributing to the organisation. I have spent this time deeply exploring the repositories, identifying bugs, and drafting issues, and I have found the community to be incredibly welcoming. I am eager to take the next step by getting involved in larger-scale projects to help make the ecosystem even better. Because of the great response from the community and my commitment, I have chosen JSON Schema as my organization for GSoC'26.

Contributions

My PRs in JSON Schema Ecosystem

PR	Description	Status
<u>#2093</u>	fixed URL wrapping	 Merged
<u>#128</u>	added new sponsors with all the details and removed the ones listed	 Merged
<u>#836</u>	Add tests for out-of-range time components in date-time format	 Merged
<u>#2139</u>	fix: changed the code blocks (JsonEditor) to look light on light theme mode	 Merged
<u>#842</u>	Add test rejecting dotted-decimal IPv4 addresses in hostname format	 Closed
<u>#877</u>	test: add some missing edge cases to json-pointer format tests	 Merged

Issues opened by me in JSON Schema Ecosystem

Issue	Description	Status
<u>#2088</u>	Bug: Long URLs don't wrap properly on JSON Schema Glossary page (Mobile View)	 Closed
<u>#2127</u>	Bug: Code blocks remain dark in Light Theme mode	 Closed
<u>#832</u>	Missing tests for out of range values in hours/minutes in date-time format	 Closed
<u>#838</u>	Add test to ensure dotted-decimal IPv4 addresses are rejected by format: hostname	 Open
<u>#2292</u>	Bug: FAQ page "+" icon appears visually off-center in accordion toggle	 Closed
<u>#859</u>	Add missing edge cases to json-pointer format tests	 Open

Qualification Task

As the qualification task, I built an annotation test harness for `networknt/json-schema-validator` (Java) — an implementation that exposes annotation data via its `OutputUnit` API but has no existing annotation test harness. The harness runs the official JSON Schema Annotation Test Suite for all draft versions (Draft 4, 6, 7, 2019-09, 2020-12).

The harness follows the universal harness contract described in this proposal.

The full implementation is available at: https://github.com/VIDIT45AGARWAL/QualificationTask_networknt

Important NOTE

A Note on Future Compatibility: This proposal is drafted with reference to the current validation test suite format. I am aware of a parallel GSoC project, "Unify the test suite", which aims to collapse per-version validation tests into a single suite using a compatibility property (similar to the approach already used by the Annotations suite). Since this may change the validation test suite format, it could introduce implementation changes or merge conflicts during the GSoC period. I am prepared to adapt this implementation plan as needed to remain compatible with the unified suite as it evolves.

Project Description

Overview

The JSON Schema Ecosystem and Bowtie

- JSON Schema is a widely used standard for describing and validating the structure of JSON data, commonly used for configuration validation and API contracts. Because JSON Schema is implemented in many programming languages—including Java, Python, Rust, Go, and JavaScript—there are dozens of independent implementations.
- Maintaining consistent behavior across these implementations is challenging. The specification is complex and has evolved across multiple drafts, which can lead to subtle differences in interpretation.
- Bowtie is the official compliance testing framework for the JSON Schema ecosystem. It evaluates implementations by running them against the community-maintained JSON Schema Test Suite and publishing the results on bowtie.report.
- Each implementation runs in a Docker-based harness that communicates with Bowtie through a lightweight JSON-lines protocol. Bowtie aggregates these results and automatically regenerates compliance reports through a GitHub Actions workflow.

The Role of Annotations in JSON Schema

- Although JSON Schema is commonly used for validation, the specification also defines annotations—metadata generated during schema evaluation. While validation produces a simple boolean result (valid or invalid), annotations provide additional semantic information about how a schema was applied.
- Examples include *"title"* and *"description"* for documentation, *"default"* for suggested values, and *"readOnly"*/*"writeOnly"* for access constraints. Some keywords also produce annotations describing which parts of the instance were evaluated.
- Annotations are particularly important for keywords such as *"unevaluatedProperties"* and *"unevaluatedItems"*, which rely on information about previously evaluated instance locations. If annotations are not collected correctly, these keywords may behave incorrectly.

The Annotation Test Suite

- To address this gap, the JSON Schema has developed a dedicated Annotation Test Suite. Unlike the validation test suite—which checks only whether instances are valid—the annotation test suite verifies whether implementations correctly produce annotation values during evaluation.
- Each test specifies the instance location, the keyword producing the annotation, and the expected annotation value. An implementation passes only if it produces the correct annotation at the correct location.

However, this test suite is not yet integrated with Bowtie, the ecosystem's primary compliance testing tool.

The Current Gap

Bowtie currently reports only validation compliance. While users can see how well implementations validate schemas, there is no visibility into annotation behavior.

This creates several challenges:

- Developers cannot determine which implementations reliably support annotation-dependent features such as form or documentation generation.
- Library maintainers lack automated regression testing for annotation behavior.
- The specification lacks systematic feedback on whether annotation semantics are consistently implementable.

Integrating the Annotation Test Suite into Bowtie would address this gap and provide the first ecosystem-wide view of annotation compliance across implementations.

Project Outcomes

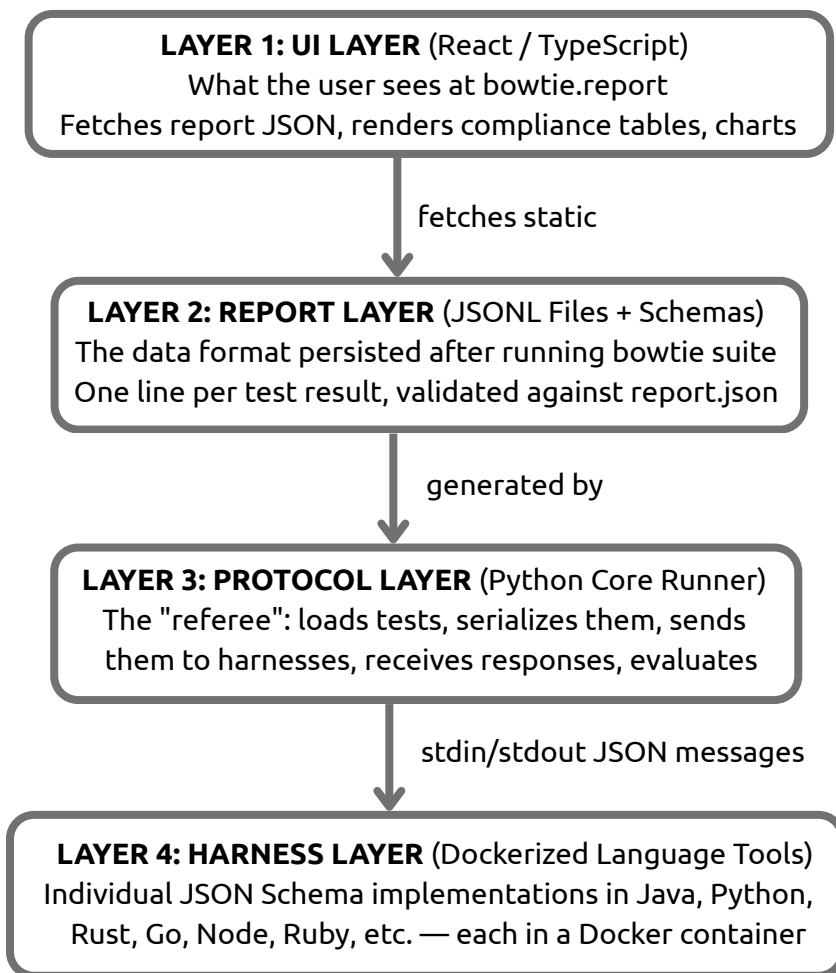
By the end of the project, the following key deliverables will be completed to provide full, end-to-end support for the Annotation Test Suite within Bowtie:

- **Protocol & Engine Upgrade:** The core Bowtie runner will be generalized to execute the official Annotation Test Suite. Rather than treating test cases simply as passing or failing strings, the system will be upgraded to securely transmit expected annotation data to the test harnesses, and accurately evaluate the complex sets of annotation values that the harnesses return.
- **Upgraded Data Model & Report Format:** The internal data model and the formal JSON report schema will be updated to store and persist annotation results. This ensures that the generated report files can represent rich metadata (like the exact keywords and values produced) alongside standard validation results.
- **Universal Harness Implementation:** Because Bowtie supports dozens of JSON Schema implementations across different languages, upgrading just one test harness is not enough. This project will establish a language-agnostic "Universal Harness Contract" for annotation collection. By the end of the project, all supported major test harnesses (e.g., Python, Rust, Go, Java) will be systematically updated to follow this pattern, allowing them to collect and emit annotation metadata.
- **New "Annotations" Reporting UI:** A dedicated, fully functional "Annotations" page will be added to the Bowtie report site. This interactive interface will visualize annotation conformance across all implementations, allowing developers to see exactly which validators correctly produce which annotations.
- **Automated CI/CD Publishing:** The automated GitHub Actions pipeline that Bowtie relies on will be extended to seamlessly handle the new test suite. Annotation reports for multiple dialects (like Draft 2019-09 and Draft 2020-12) will be generated and published continuously to the live bowtie.report website, ensuring the community always has access to the most up-to-date compliance data.

Implementation Plan

Bowtie's architecture can be thought of as four distinct, collaborating layers — each one only knowing about its immediate neighbour. A message about a test starts at the top (UI), travels down through the Report layer and Protocol layer, and finally reaches a Harness (the individual language tool). The response travels back up.

To add Annotation support, **every single layer** needs changes. This plan describes every change in the order a message travels through the system.



Currently, only **validation** travels through all 4 layers. An annotation test is structurally incompatible with every single layer. The following plan fixes this, layer by layer, bottom-to-top.

Layer 4 — The Harness Layer: Universal Annotation Contract

Goal: Every language implementation must be able to collect annotation metadata and emit it back to the runner in a standardized format.

Background: What a Harness Does Today

A Bowtie test harness is a Docker container that speaks a simple JSON-lines protocol. The runner sends it a message like this:

```
{
  "cmd": "run",
  "seq": 1,
  "case": {
    "description": "integer is valid",
    "schema": { "type": "integer" },
    "tests": [
      { "description": "an integer", "instance": 1 }
    ]
  }
}
```

The harness validates the instance against the schema and responds:

```
{
  "seq": 1,
  "results": [
    { "valid": true }
  ]
}
```

That's the entire current contract. A simple boolean. **Annotation tests break this** entirely — they require the harness to return a rich tree of metadata, not a single boolean.

The Problem: Annotations Are Per-Keyword, Per-Location

The JSON Schema specification states that when a keyword like *"title"* is evaluated, it produces an **annotation** — a value that says: "At this location in the instance, this keyword produced this value, and it came from this part of the schema."

For example, given the schema:

```

{
  "type": "object",
  "title": "User",
  "properties": {
    "name": { "title": "Full Name" }
  }
}

```

When the instance `{ "name": "Vidit" }` is evaluated, it produces these annotations:

- At instance root `"",` keyword `"title"` from schema location `"#"` → value `"User"`
- At instance path `"/name",` keyword `"title"` from schema location `"#/properties/name"` → value `"Full Name"`

A harness must be able to collect all of this and send it back in a nested structure:

```

{
  "": {
    "title": { "#": "User" }
  },
  "/name": {
    "title": { "#/properties/name": "Full Name" }
  }
}

```

This is the annotations map: *instanceLocation* → *keyword* → *schemaLocation* → *value*.

The Solution: The Universal Harness Contract

I will establish a language-agnostic 4-step contract that every harness must implement.

Important Note: The code snippets below use Java (the *networknt* JSON Schema Validator) purely as a reference implementation to demonstrate the pattern, as I have already prototyped this in the qualification task. During the GSoC coding period, I will translate this exact logical pattern into the native, idiomatic mechanisms of every major language harness.

Step 1 — Detect Annotation Tests at Runtime

The most important design decision: **do not force all tests to collect annotations**. Annotation collection via structured output trees is expensive. A harness should only pay this cost when the test actually requires it. The harness checks a single condition at the start of each test case:

```
boolean hasAssertions = false;
if (tests.isArray()) {
  for (JsonNode test : tests) {
    if (test.has("assertions") && test.get("assertions").isArray() && test.get("assertions").size() > 0) {
      hasAssertions = true;
      break;
    }
  }
}
```

Why this per-case check matters: The standard validation test suite sends hundreds of tests per dialect. If every test triggered expensive annotation collection, it would make the standard test runs dramatically slower. The per-case branch guarantees the validation path is completely unaffected — it runs at identical speed with identical results.

Step 2 — Execute in Hierarchical Mode

When *hasAssertions* is *true*, the harness switches to **hierarchical/tree output** mode instead of the fast boolean flag mode:

```
if (hasAssertions) {
  // Expensive path: collect full annotation tree
  SchemaValidatorsConfig config = new SchemaValidatorsConfig();
  config.setAnnotationCollectionEnabled(true);
  config.setAnnotationCollectionFilter(keyword -> true); // Collect ALL keywords

  JsonSchema schema = factory.getSchema(schemaNode, config);
  OutputUnit outputUnit = schema.validate(instanceNode, OutputFormat.HIERARCHICAL);

  // Walk the tree, collect all annotations
  Map<String, Map<String, Map<String, Object>>> annotations = new HashMap<>();
  collectAnnotations(outputUnit, annotations);

  emit(new TestResult(outputUnit.isValid(), annotations));
} else {
  // Fast path: unmodified standard validation – ZERO REGRESSION
  Set<ValidationMessage> errors = schema.validate(instanceNode);
  emit(new TestResult(errors.isEmpty(), null));
}
```

Step 3 & 4 — Recursively Walk the Tree and Normalize Paths

The hierarchical output is a deeply nested tree. I wrote a  **collectAnnotations()** recursive function that walks every node, extracts keyword annotations, normalizes the paths seamlessly, and builds the flat *instanceLocation* → *keyword* → *schemaLocation* → *value* map needed by Bowtie:

```
private void collectAnnotations(
    OutputUnit unit,
    Map<String, Map<String, Map<String, Object>>> annotations
) {
    if (unit == null) return;

    String instanceLoc = unit.getInstanceLocation() != null
        ? unit.getInstanceLocation().toString() : "";

    // Normalize schema location formatting to universally match Annotation Test Suite
    String schemaLoc = unit.getEvaluationPath() != null
        ? unit.getEvaluationPath().toString() : "";
    if (schemaLoc.isEmpty()) {
        schemaLoc = "#";
    } else if (schemaLoc.startsWith("/")) {
        schemaLoc = "#" + schemaLoc;
    }

    // Extract this node's annotations (each keyword produces a value)
    Map<String, Object> unitAnnotations = unit.getAnnotations();
    if (unitAnnotations != null && !unitAnnotations.isEmpty()) {
        for (Map.Entry<String, Object> entry : unitAnnotations.entrySet()) {
            annotations
                .computeIfAbsent(instanceLoc, k -> new LinkedHashMap<>())
                .computeIfAbsent(entry.getKey(), k -> new LinkedHashMap<>())
                .put(schemaLoc, entry.getValue());
        }
    }

    // Recurse into all child nodes
    if (unit.getDetails() != null) {
        for (OutputUnit child : unit.getDetails()) {
            collectAnnotations(child, annotations);
        }
    }
}
```

This normalization is the difference between a test passing and failing. Without uniform paths, the runner would compare `#/properties/name` against `/properties/name` and report a mismatch even when the value is correct.

Layer 3 — The Protocol Layer: The Python Core Runner

Goal: The runner must be updated to (1) correctly transmit annotation test cases to harnesses, (2) correctly receive and store annotation results, and (3) correctly evaluate whether a harness passed or failed an annotation test.

Background: How the Runner Communicates

The Python runner is the "referee". It is responsible for:

- Loading test cases from disk
- Serializing them and sending them to harnesses via stdin
- Reading responses from harnesses via stdout
- Comparing responses against expected results
- Writing the final JSONL report

All communication is JSON-lines. The runner parses what the harness sends back using `json.loads()` and maps it to Python data objects. This is where annotation data needs to flow through.

Change 1 — The "Denylist" Protocol Fix (🐍_core.py)

The Problem: Annotations Are Silently Stripped

When the runner serializes a test case to send to a harness, it deliberately removes the expected results. The logic is: "Don't let the harness see the right answer." Currently, it does this with an **allowlist approach** — it keeps only a fixed set of known fields:

```
# BEFORE: Allowlist approach – only these fields survive
allowed = {"description", "comment", "instance", "schema"}
stripped_test = {k: v for k, v in test.items() if k in allowed}
```

The problem is obvious: *assertions* is not in the allowlist, so it is **silently deleted before it ever reaches the harness**. This means every harness always falls into the validation path and never knows it was supposed to collect annotations.

The Fix: Switch to a Denylist

The solution is to invert the logic. Instead of specifying what to keep, specify only what to remove:

```
# AFTER: Denylist approach – only strip the 'valid' flag and null comments
# ALL other fields – including future ones – automatically pass through
serializable["tests"] = [
    {
        k: v
        for k, v in test.items()
        if k != "valid"                    # Strip the expected answer
        and not (k == "comment" and v is None) # Strip null comments (clutter)
        # "assertions", "description", "instance", and any future fields
        # survive automatically – no code changes needed for future extensions
    }
    for test in serializable.pop("tests")
]
```

Change 2 — Building the Evaluation Engine (🐘_commands.py)

The Problem: The Runner Cannot Compare Annotation Results

The runner currently evaluates whether a test passed like this:

```
# BEFORE: Only handles boolean validation outcomes
passed = result.valid == test.expected_valid
```

This is completely incompatible with annotation tests. An annotation test's expected value is not *True* or *False* — it is a list of assertion objects like:

```
[
  {
    "location": "/name",
    "keyword": "title",
    "expected": { "#/properties/name": "Full Name"}
  }
]
```

Fix 1: Expand the 🐘 *TestResult* Data Class

First, the Python data class that represents a harness response must be updated to hold annotations:

```
# Before
@frozen
class TestResult:
    valid: bool

# After
@frozen
class TestResult:
    valid: bool
    annotations: dict[str, dict[str, dict[str, Any]]] | None = field(default=None)

    @classmethod
    def from_dict(cls, data: dict) -> "TestResult":
        """Deserialize from harness JSON response"""
        data_copy = dict(data)
        valid = data_copy.pop("valid", False)
        annotations = data_copy.pop("annotations", None) # NEW
        return cls(valid=valid, annotations=annotations)
```

Fix 2: Polymorphic `matches()` Method

Instead of a direct equality check, I introduce a `matches()` method that handles both test types intelligently:

```
def matches(self, expecting: Any) -> bool:
    """
    Evaluate if this result passes the given expectation.

    - If expecting is a bool → standard validation test
    - If expecting is a list → annotation test (subset equality check)
    - If expecting is None → no expectation, always passes (unknown)
    """
    if expecting is None:
        return True # No expectation means we have no basis to call it a failure

    if isinstance(expecting, bool):
        # STANDARD VALIDATION PATH – unchanged behavior
        return self.valid == expecting

    # ANNOTATION PATH – a list of assertions to check
    if self.annotations is None:
        return False # Harness returned no annotations but the test expects some

    for assertion in expecting:
        location = assertion.get("location", "") # Where in the instance
        keyword = assertion.get("keyword", "") # Which schema keyword
        expected_values = assertion.get("expected", {})

        # Decode any URL-encoded characters in the expected schema locations
        # e.g. "%2F" becomes "/" – critical for property names containing slashes
        decoded_expected = {
            urllib.parse.unquote(k): v
            for k, v in expected_values.items()
        }

        # Navigate: annotations[location][keyword] should equal expected_values
        actual = self.annotations.get(location, {}).get(keyword, {})
        if actual != decoded_expected:
            return False

    return True # All assertions matched
```

Why subset equality and not full equality? A test case only asserts what it cares about. An implementation might produce correct annotations for *"title"* AND also produce annotations for *"properties"* — extras that aren't being tested. Requiring exact equality would cause false failures. Subset equality ("does the actual output contain at least what was expected?") is the correct interpretation of the test suite.

Change 3 — Auto-Detecting the Annotation Suite (`_suite.py`)

The Problem: Different Test Suite Format

The standard validation test suite and the annotation test suite have the following key differences in their file formats:

1. Top-level is an object with a `"suite"` key, not an array
2. Tests use `"instance"` instead of `"data"`
3. Tests have `"assertions"` instead of `"valid"`
4. Cases carry a `"compatibility"` field (the minimum dialect required)
5. Individual tests may lack a `"description"` field

The Fix: Auto-Detection + New Loader Function

Rather than requiring a command-line flag from the user, Bowtie will automatically detect which suite format it is given:

```
def _detect_suite_type(path: Path) -> str:
    """
    Peek at the first JSON file to determine suite format.
    Returns "annotation" or "validation".
    """
    sample_path = next(path.glob("*.json"), None)
    if sample_path is None:
        return "validation" # Default

    content = json.loads(sample_path.read_text())

    # Annotation suites are a top-level object with a "suite" key
    # Validation suites are a top-level array
    if isinstance(content, dict) and "suite" in content:
        return "annotation"
    return "validation"
```

A new  `annotation_cases_from()` generator function handles the entire loading process:

```
def annotation_cases_from(
    paths: Iterable[Path],
    dialect: Dialect,
) -> Iterable[TestCase]:
    """Load and yield TestCase objects from annotation test suite files."""

    for path in paths:
        content = json.loads(path.read_text())
        if not isinstance(content, dict) or "suite" not in content:
            continue

        for case in content["suite"]:
            # Filter: skip test cases incompatible with the current dialect
            # e.g., don't run Draft 2020-12-only tests against a Draft 7 harness
            compatibility = case.pop("compatibility", None)
            if compatibility:
                required_dialect = Dialect.by_alias().get(compatibility)
                if required_dialect and required_dialect > dialect:
                    continue # This test needs a newer dialect than requested

            # Auto-generate descriptions for tests that are missing them
            # (the annotation suite sometimes omits test-level descriptions)
            for i, test in enumerate(case.get("tests", [])):
                if "description" not in test:
                    if test.get("assertions"):
                        a = test["assertions"][0]
                        test["description"] = (
                            f"instance at {a['location']}!r} "
                            f"keyword {a['keyword']}!r}"
                        )
                    else:
                        test["description"] = f"test {i + 1}"

            yield TestCase.from_dict(dialect=dialect, **case)
```

Layer 2 — The Report Layer: Formal Schema Definitions

Goal: Every message in Bowtie is validated against a formal JSON Schema before being used. All three relevant schemas must be updated to recognize annotation data as valid.

Background: Why Schemas Matter Here

Bowtie uses JSON Schema to validate its own data structures. There are three schemas relevant to this change:

Schema File	What it validates
 bowtie/schemas/models/test.json	The format of a single test case SENT to harnesses
 bowtie/schemas/io/commands/run.json	The format of a harness RESPONSE
 bowtie/schemas/report.json	The format of the final JSONL report file

Change 1 — `{}` `models/test.json`

Before: A test case only had *description*, *comment*, *instance*, and *valid*.

After: Add *assertions* as an optional array property

Change 2 — `{}` `io/commands/run.json`

Before: A harness response only accepted *valid: boolean*.

After: Add an optional *annotations* map

```
{
  "properties": {
    "valid": { "type": "boolean" },
    "annotations": {
      "type": "object",
      "description": "Keyed by instance location (JSON Pointer)",
      "additionalProperties": {
        "type": "object",
        "description": "Keyed by keyword name (e.g. 'title')",
        "additionalProperties": {
          "type": "object",
          "description": "Keyed by schema location URI",
          "propertyNames": { "format": "uri-reference" }
        }
      }
    }
  },
  "required": ["valid"]
}
```

Why valid remains required: Even in an annotation test, the harness must still report whether validation passed or failed. The two are complementary, not mutually exclusive.

Change 3 — `{}` `report.json`

Before: The expected array in a result only allowed *boolean* or *null*

After: Also allow array and object to hold annotation assertions:

Why not create a separate report schema for annotations? Maintaining a single, unified schema is significantly simpler for all downstream tooling that reads Bowtie report files. The runtime type check (*test.assertions !== undefined*) in the frontend handles the interpretation correctly.

Layer 1 — The UI Layer: React Frontend

Goal: Create a new dedicated "Annotations" page on bowtie.report that visualizes annotation compliance with the same clarity and interactivity as the existing validation page.

Background: How the UI Currently Works

The frontend is a single-page React application built with TypeScript and Vite. All the compliance data is fetched as static JSON files from `bowtie.report` at runtime. There is no backend — the React app fetches pre-generated report files and renders them.

The existing validation flow:

- User navigates to `bowtie.report`
- React app fetches `bowtie.report/draft2020-12.json`
- `parseReportData.ts` deserializes the JSONL file into typed objects
- Components like `CasesSection`, `ImplementationCard` render the data

For annotations, the exact same flow applies — but the data structure is different.

Phase 1: What Can Be Reused

The existing frontend is well-architected with presentation components cleanly separated from data parsing. I will heavily reuse these existing components to guarantee visual consistency and minimize duplicate code:

1. Layout Components (`NavBar`, `Summary`, `Cases`, `ImplementationReportView`) :

- These components are highly decoupled. They accept agnostic props and do not care if the underlying data is a validation result or an annotation result. I will reuse these directly on the new Annotations page.

2. Data Fetching Infrastructure (`Dialect.ts`):

- I will extend the existing `Dialect` class by adding an `annotationsRoutePath` property and a  `fetchAnnotationReport()` method.

Phase 2: What Needs To Be Built (Net-New)

Because comparing annotation trees is fundamentally different from comparing boolean validation flags, I will build the following net-new components:

Polymorphic Result Parsing (`parseReportData.ts`)

- The current parsing logic assumes every test yields a boolean. I will write a branching parser: if `test.assertions` is present, it delegates to a new  `matchesAnnotations()` algorithm.
- **Algorithm Design:**  `matchesAnnotations()` will apply **subset equality**, not strict equality. A test only asserts what it cares about. If a harness emits correct annotations for `"title"` but also emits extra annotations for `"properties"`, it should still pass.

```
function matchesAnnotations(expected: ExpectedAssertion[], actual: HarnessAnnotations): boolean {
  return expected.every(assertion => {
    const actualValue = actual[assertion.location]?.[assertion.keyword];
    // Deep subset equality check
    return isSubset(assertion.expected, actualValue);
  });
}
```

The Top-Level Dashboard (`AnnotationReportView.tsx`)

- A new primary view component registered in  `App.tsx` at the `route / annotations/:draftName`.
- It will orchestrate the data fetch, invoke the new parser, and map the results into the reused layout components ( `SummarySection`, etc.).

The Assertion Diff Renderer (`AnnotationComparison.tsx`)

- Currently, a failing validation test just renders a red "Failed" badge. A failing annotation test requires much deeper context.
- I will build a dedicated, rich UI component that renders a diff table when an annotation test fails. It will visually contrast the  `expected` schema locations and values against the `actual` nested tree emitted by the harness, making it immediately obvious to library authors exactly why their harness failed the test.

Cross-Cutting Concern: CI/CD Pipeline (`report.yml` + `ui.yml`)

Goal: Annotation reports must be continuously regenerated and deployed automatically, just like validation reports.

How Reports Are Currently Generated

Every 6 hours, a GitHub Actions workflow (*report.yml*) runs:

- Reads all supported dialects from `{ } data/dialects.json`
- Runs *bowtie suite* against all harnesses for each dialect
- Saves the JSONL output (e.g., `{ } draft2020-12.json`)
- Calls *ui.yml* which builds the React app and deploys everything to GitHub Pages

The Change: Add a Parallel Annotation Job

I will add a new job to *report.yml* that mirrors this pattern for the annotation suite:

```
regenerate-annotation-reports:
  runs-on: ubuntu-latest
  needs: dialects
  strategy:
    fail-fast: false
  matrix:
    # Only dialects that support annotation collection
    version: ["draft2020-12", "draft2019-09"]

  steps:
    - uses: actions/checkout@v6

    # Pull the annotation test suite from the JSON Schema spec repo
    - name: Checkout Annotation Test Suite
      uses: actions/checkout@v6
      with:
        repository: json-schema-org/json-schema-spec
        path: annotation-suite

    - name: Install Bowtie
      uses: ./

    # Run every implementation against the annotation suite for this dialect
    - name: Generate Annotation Report
      run: |
        bowtie suite \
          $(bowtie filter-implementations | sed 's/^/-i /') \
          annotation-suite/annotations/tests/${{ matrix.version }}/ \
          > annotations-${{ matrix.version }}.json

    - uses: actions/upload-artifact@v6
      with:
        name: annotation-report-${{ matrix.version }}
        path: annotations-${{ matrix.version }}.json
```

Then, in the *site* assembly job, the annotation report artifacts are downloaded alongside validation reports:

```
- name: Include Annotation Reports
  uses: actions/download-artifact@v7
  with:
    pattern: annotation-report-*
    path: site/annotations/ # → deployed to bowtie.report/annotations/
    merge-multiple: true
```

Once deployed, the frontend's `fetchAnnotationReport()` will automatically find the files at <https://bowtie.report/annotations/draft2020-12.json> — no frontend changes are needed for this step.

Verification & Testing Strategy

Goal: Maintain 100% stability of the core validation engine while ensuring the new annotation logic is robust and correct.

1. Unit & Property-Based Testing (Runner Core):

- *Denylist Logic*: Exhaustive testing of the registry and serialization layer to verify that assertions are preserved while `expected` results are correctly stripped.

2. Integration & Consistency Testing (End-to-End):

- *Mock Annotation Harness*: I will create a reusable Python-based mock harness that can simulate various annotation behaviors (including "extra" annotations and missing ones) to verify the runner's evaluation logic.
- *Regression Testing*: I will run the existing validation test suite across all dialects to ensure my changes to the protocol layer have **zero impact** on standard validation performance or correctness.

3. Schema Validation Testing:

- I will add tests specifically for Bowtie's meta-schemas (*report.json*, *test.json*). I will feed them both valid and "maliciously malformed" annotation payloads to verify that the formal schema validation correctly catches protocol violations.

4. Frontend Component & Algorithm Tests:

- `matchesAnnotations`: Unit tests for the subset equality algorithm covering nested objects, arrays, and primitive values.
- *View State Testing*: Using Vitest and React Testing Library to verify that the `AnnotationReportView` handles edge cases like missing dialects, network failures, and extremely large annotation trees without crashing.

Documentation

Goal: Ensure the new functionality is discoverable, well-documented for end-users, and provides a clear integration path for harness authors.

- **Harness Authorization Guide:** I will create a new guide in the Bowtie documentation (or update the existing implementation guide) that explicitly describes the "Universal Harness Contract" for annotations. This will include code examples in multiple languages (Python, Java, Rust) to help implementation authors add support to their own harnesses.
- **User-Facing "How-To" for Annotations:** I will add a section to the user documentation explaining how to run the annotation suite using the CLI and how to interpret the results on the new dashboard.

Harness Expansion Strategy: Implementation Readiness

To ensure the systematic rollout of the Universal Harness Contract across the Bowtie ecosystem, I have categorized the existing implementations into **2 Tiers**. This strategy guarantees early pipeline validation by targeting libraries that natively support annotations first, before moving to the ones that don't.

Tier	Characteristics	Target Implementations
Tier-1	Supports the annotation test suite already (10+)	cpp-blaze, java-com.networknt-json-schema-validator, js-hyperjump-json-schema, perl-JSON-Schema-Modern, python-jschon, lua-schema, rust-jsonschema, swift-jsonschema, java-json-schema, json-everything, ruby-json_schemer
Tier-2	Doesn't support the annotation test suite currently	rest harnesses present in bowtie ecosystem

Timeline

Community Bonding Period	
May 1 – May 24	• Goal: Reach a full technical consensus with mentors on the implementation details. • Discuss the "Universal Harness Contract" design with mentors and the JSON Schema community. • Finalize the local development environment (Docker, React, Python) and the existing Java/Python proof-of-concepts. • Refine the draft schemas in bowtie/schemas/ to ensure they cover all necessary annotation metadata. • Outcome: A finalized technical blueprint and a set of verified local test harnesses.
Coding Phase: 12-Week Schedule	
Week 1: May 25 – May 31	• Update models/test.json to define the assertions property • Update io/commands/run.json for the annotations response • Relax report.json expected types.
Week 2: June 1 – June 7	• switch from allowlist to denylist in _core.py for test serialization. • Expand TestResult in _commands.py • implement the polymorphic matches() method • implement auto-detection in _suite.py.
Week 3: June 8 – June 14	• Finalize the Java networknt harness as the standard reference harness
Week 4: June 15 – June 21	• Target some Tier1 harnesses like js-hyperjump, jschon, json-everything and json-schema-modern
Week 5: June 22 – June 28	• Submit the project for mid-term Evaluation • Target some more Tier1 harnesses like blaze, rust-jsonschema and swift-jsonschema.
Week 6: June 29 – July 5	• Target the remaining Tier1 harnesses - Ruby json_schemer, lua-schema, java-json-schema

Week 7: July 6 - July 12	- Implement <code>fetchAnnotationReport()</code> and the <code>matchesAnnotations</code> subset equality algorithm - Build the <code>AnnotationReportView.tsx</code> dashboard and summary grids analyzing the 11 updated harnesses.
Week 8: July 13 - July 19	- Build the <code>AnnotationComparison.tsx</code> diff renderer within <code>CasItem.tsx</code> to show specific pass/fail reasonings - implement interactive debugging tools.
Week 9: July 20 - July 26	- Update <code>report.yml</code> to continuously generate the new reports so the React dashboard renders live output. - Verify cross-language consistency between all newly supported harnesses.
Week 10: July 27 - August 2	- Implement Hypothesis-based property testing to mathematically verify the core runner. - Finalize the complete Harness Implementation Guide and developer documentation to ensure future contributors can easily interact with the annotation system.
Week 11: August 3 - August 9	- Address any overflowing tasks or edge-case bugs from the core pipeline - Begin Phase 2: Start updating Tier-2 harnesses (implementations that don't natively support annotations yet). Note that these harnesses will initially fail most annotation tests — this is expected and informative, as it establishes a compliance baseline for the community rather than producing immediately passing results
Week 12: August 10 - August 16	Continue Tier-2 harness updates. Since these implementations lack native annotation support, the goal here is infrastructure readiness — ensuring each harness correctly communicates with the annotation pipeline, so that as upstream libraries add annotation support, Bowtie can immediately reflect that progress without further harness changes
Final Week: (Submission) August 17 - August 24	- Submit the final work product, code samples, and mentor evaluations. - Prepare a public summary of all contributions and open-source impact for the Bowtie community. - Formalize the Post-GSoC roadmap, explicitly tracking remaining Tier-2 harnesses as upstream PR targets with clear completion criteria

Post GSoC Plans

- **Complete Remaining Harness Updates:** Systematically update any leftover JSON Schema test harnesses (most probably tier-2) in Bowtie that were not covered during the core Summer timeline, ensuring 100% ecosystem coverage.
- **Annotation Suite Benchmarking:** Implement benchmarking infrastructure for the Annotation Test Suite to quantify the exact time and memory performance overhead of annotation collection across all language implementations. This will be an invaluable metric for production users.
- **Continuous Community Engagement:** Remain an active, long-term contributor to the JSON Schema ecosystem, help maintain Bowtie, and guide/mentor future GSoC contributors based on my own experiences.